

Teaching Exam

Subject – Computer Science

**Data Structure and
Algorithm**



Lecture No -2

By-Satyendra Chaudhary Sir

Topics to be Covered

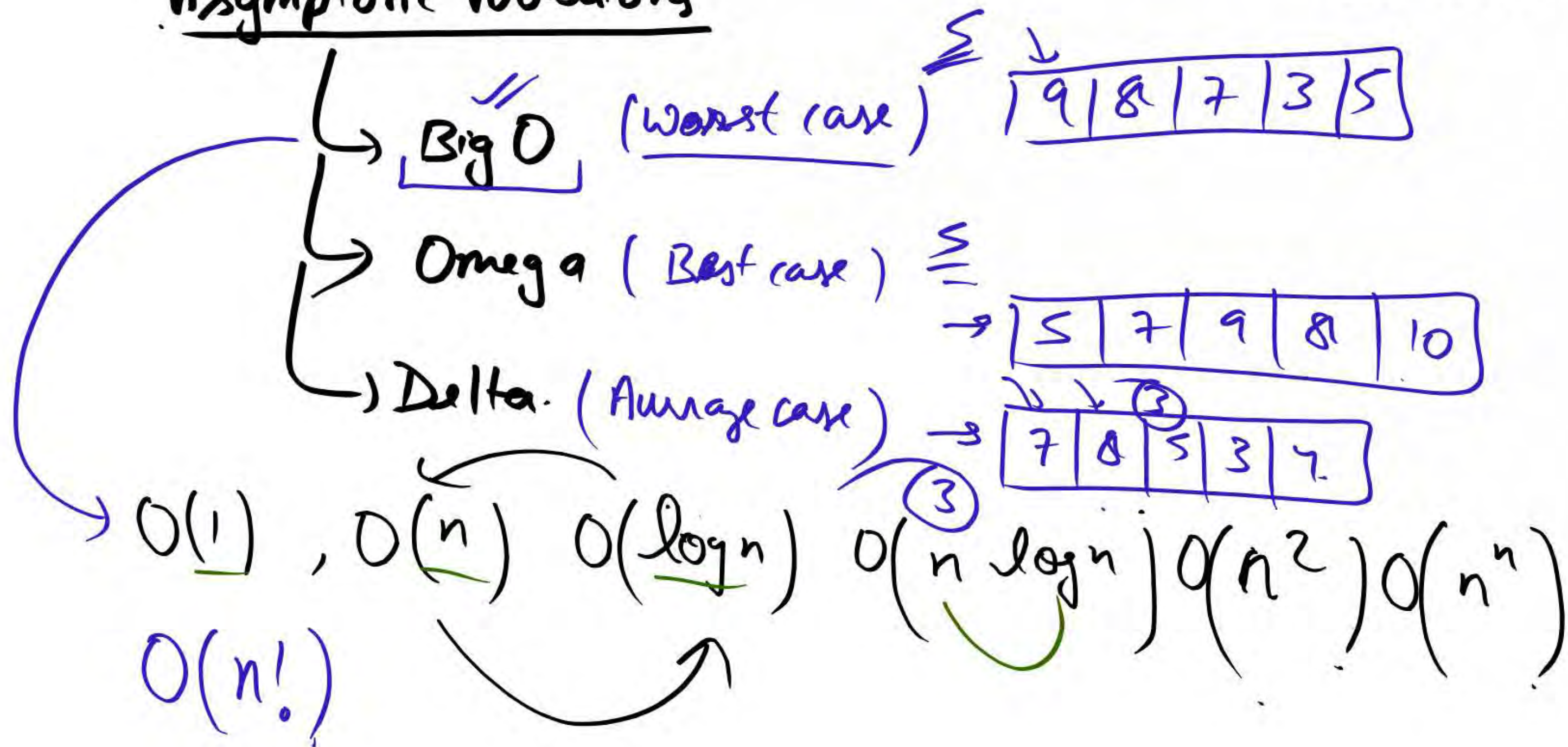


Topic

Complexities =

→ Data Structure

Asymptotic Notations



Primitive Data Types //

- Definition: Basic data types provided by a programming language.
- Examples:
 - Integer
 - Float
 - Character
 - Boolean
- Characteristics:
 - Predefined
 - Simple operations

memory size predefined

Non-Primitive Data Types

- Definition: Data types derived from primitive types.

→ constructed by Primitive data types.

- Examples:

- Arrays
- Strings
- Structures
- Classes

→ Derived Data Types. (Predefined by compiler)

→ by Primitive data types and derived data types.

→ User defined data types. (not predefined, user defined)

Linked list

Tree

- Characteristics:

- Can store multiple values
- User-defined or derived

: Linear Data Structures

- Definition: Elements arranged sequentially or linearly.

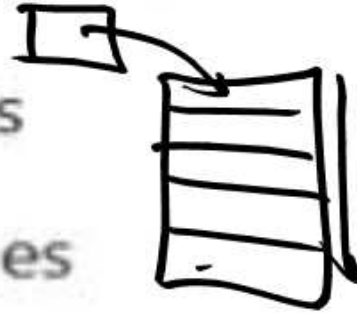
- Examples:  Data Structures are talked on 2 level.

✓ Arrays

✓ Linked Lists

✓ Stacks

✓ Queues



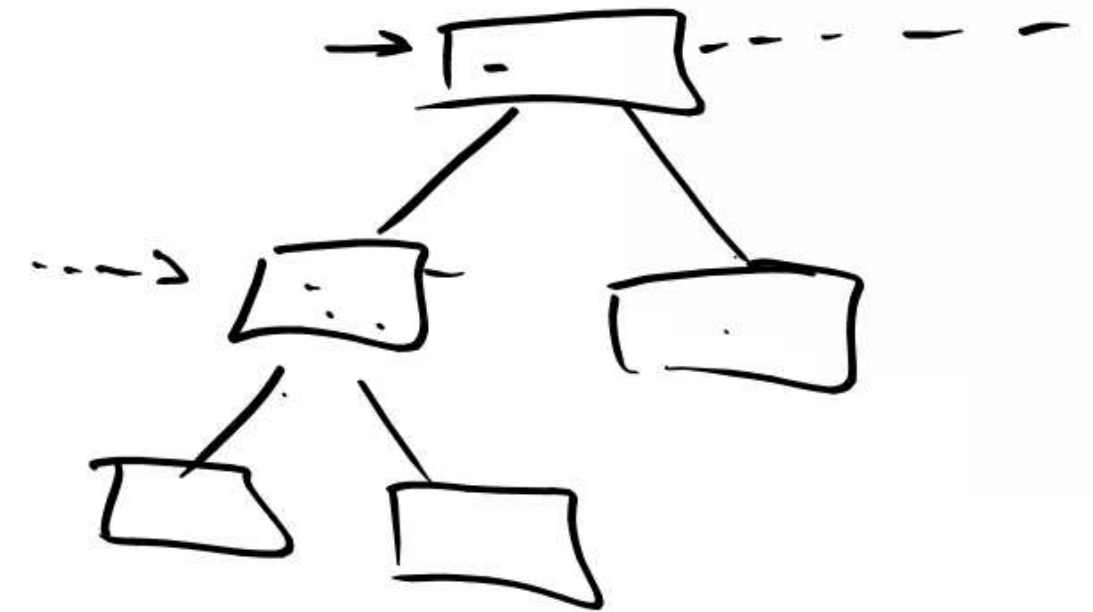
1) Abstraction level.

2) Memory level.



- Characteristics:

- Single level of elements
- Easy traversal



Non-Linear Data Structures

- Definition: Elements are not stored sequentially.
- Examples:
 - Trees
 - Graphs
- Characteristics:
 - Hierarchical relationships
 - Complex traversal

main source → Array, Linked List

: Homogeneous Data Structure

- Definition: All elements are of the same data type.
- Example: Arrays of integers, lists of floats .
- Benefit: Easier to manage and optimize memory

int a[10] •
0 9
| | | | |

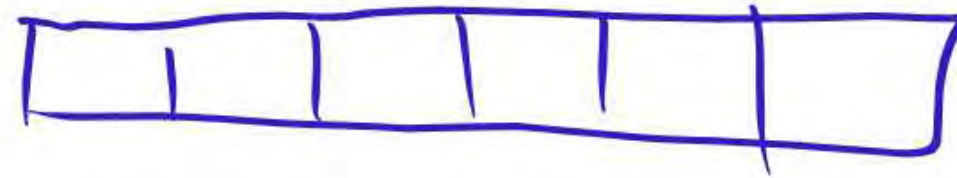
Non-Homogeneous Data Structure

- Definition: Elements can be of different data types.
- Example: Structures (struct), Classes, Tuples, List.
- Benefit: More flexible for complex data representation.

Array

`int a[20];` → C
`int[] score = new int[20];` ← Java
`int[] score = {1, 2, 3, 4, 5};`

- Definition: Collection of elements stored in contiguous memory locations
- Characteristics: `int a[] = { - - - };`



- Homogeneous

- Fixed size

- Random access using indices

`int a[20]`

`a[s] = 25;` $O(1)$



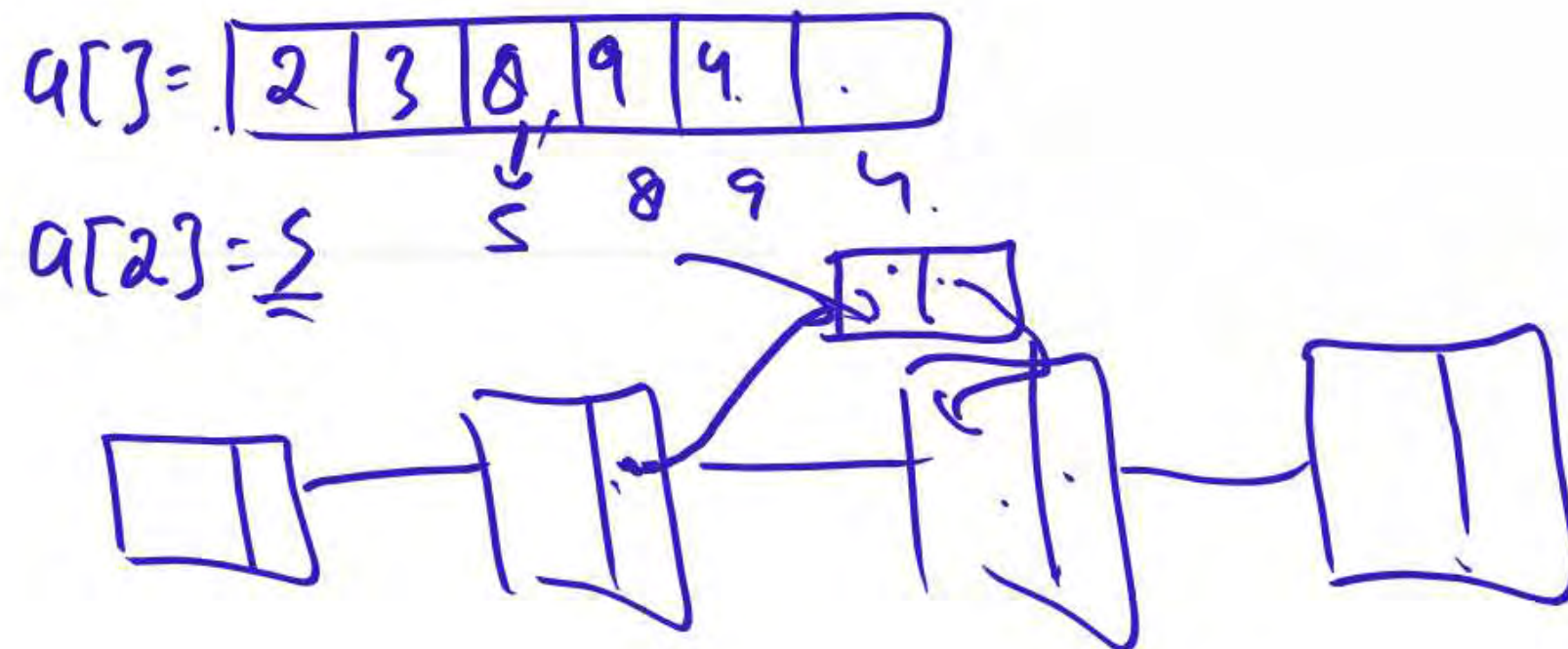
: Advantages of Arrays

- Fast access using indices
- Easy to implement and use
- Efficient for static data storage

Disadvantages of Arrays

- Fixed size (cannot grow dynamically)
- Insertion/deletion is costly
- Wastes memory if not fully used

int a[50] =



Size of an Array

26th June
x

30th June.

$$(30 - 26) + 1$$

$$\frac{1}{7} \quad \frac{7}{(7-1)+1}$$

- Formula: Total Size = Number of Elements x Size of each Element

- Example:

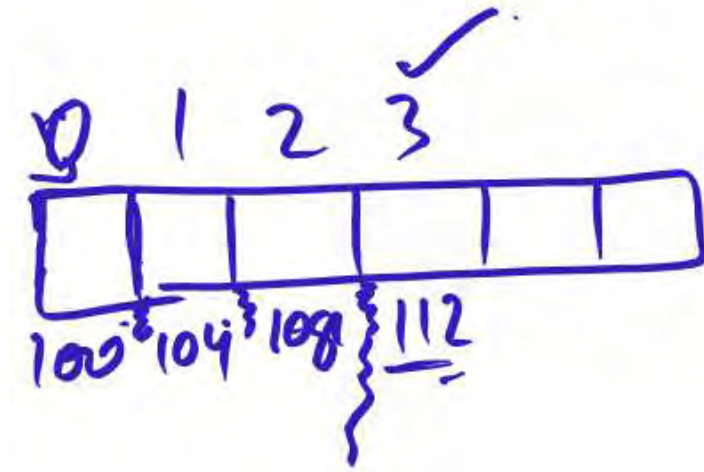
a[0] a[1] a[2] a[3] a[4]

- int arr[5]; (Assuming 4 bytes for int)

- Size = 5 x 4 = 20 bytes

a[2] a[3] 4 5 6
(6-2)+1

: Address of Element at kth Index (1D Array)



• Formula:

$$\bullet \text{ Address} = \text{Base Address} + \underline{k} * \text{Size of Data Type}$$

• Example:

$$\bullet \text{ Base Address} = \underline{1000}$$

$$\bullet \underline{k} = 3, \text{ Size} = \underline{4 \text{ bytes}}$$

$$\bullet \text{ Address} = 1000 + 3 * 4 = \underline{1012}$$

2D Array

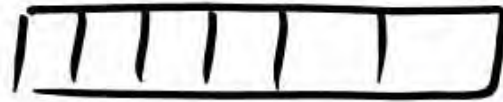
row column
`int a[5][3] =`

`a[0][2] = 5`

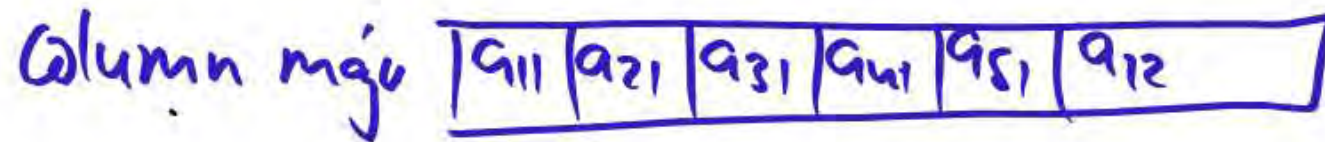
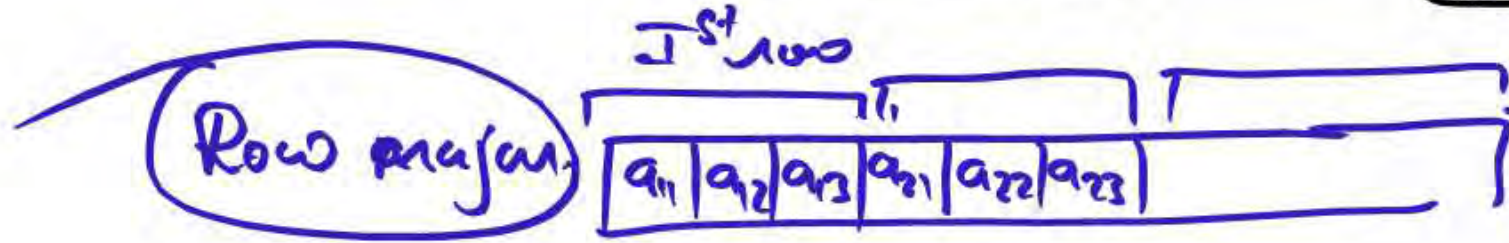
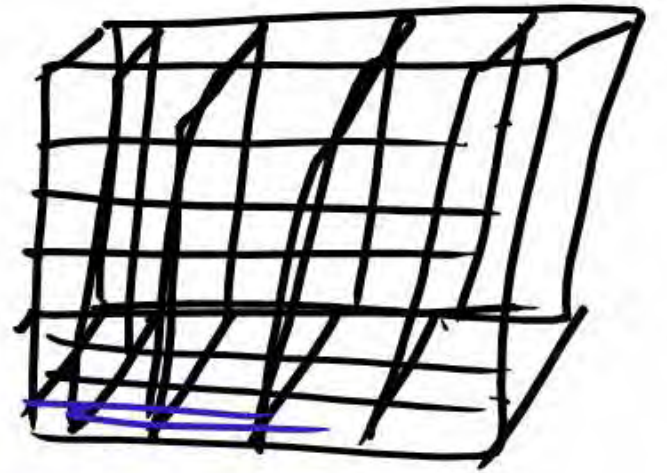
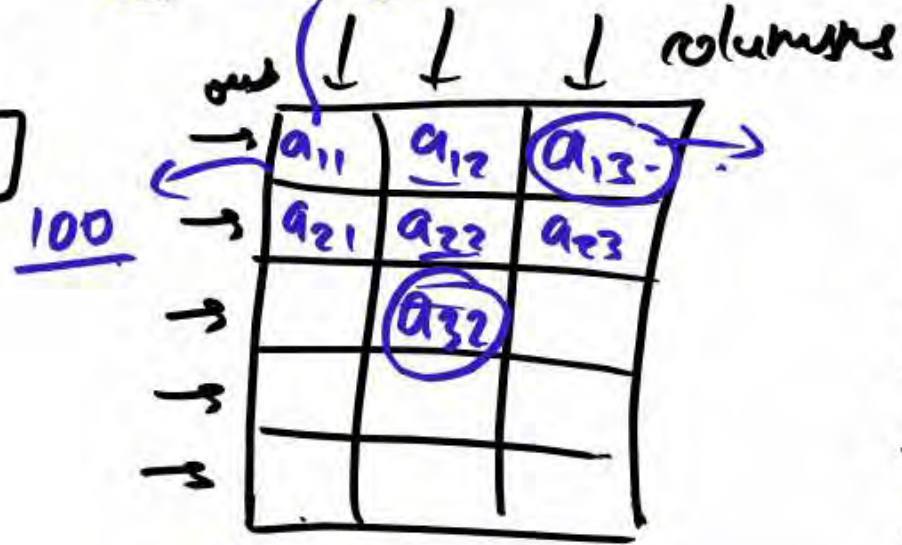
memory location

`int a[][] = { {1, 2, 3}, {3, 2, 1}, {1, 0, 1}, {4, 1, 3} };`

Abstraction level



(1-indin) byte



Address Calculation in 2D Array (Row-Major Order)

Formula:

$$\text{Address} = \text{Base} + [i * \text{columns} + j] * \text{Size}$$

Example:

$$\text{Base} = 2000, i = 1, j = 2, \text{columns} = 4, \text{Size} = 4$$

$$\text{Address} = 2000 + [(1 * 4 + 2) * 4] = 2000 + 24 = 2024$$

$$2000 + [(1-0)(4) + (2-0)](4+2)4 =$$

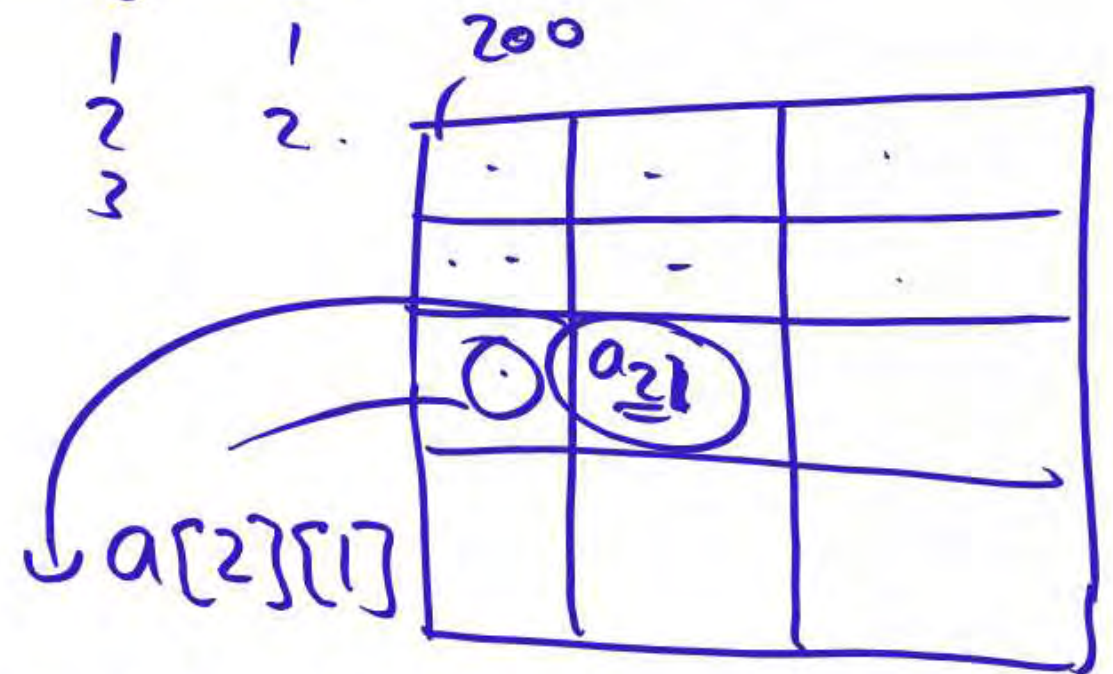
$$\underline{2024}$$

no of rows
no of columns

$$a[4][3] =$$

4 bytes

0 0
1 1
2 2
3 3



$$200 + 4 * [(2-0)(3) + (1-0)]$$

$$\underline{228}$$

: Address Calculation in 3D Array (Row-Major Order)

- Formula:
 - $$\text{Address} = \text{Base} + [(i * Y * Z) + (j * Z) + k] * \text{Size}$$
- Example:
 - Base = 3000, i=1, j=1, k=2, Y=3, Z=4, Size=4
 - $$\text{Address} = 3000 + [(134) + (14) + 2] * 4 = 3000 + 624 = 3248$$

1. Which of these best describes an array?

- a) A data structure that shows a hierarchical behavior
- b) Container of objects of similar types -
- c) Arrays are immutable once initialised -
- d) Array is not a data structure

2. How do you initialize an array in C?

a) int arr[3] = (1,2,3);^x

b) int arr(3) = {1,2,3};[↙]

✓ c) int arr[3] = {1,2,3}; ✓
[↓] int arr[3] = {1,2,3};

d) int arr(3) = (1,2,3);

3. How do you instantiate an array in Java?

a) `int arr[] = new int(3);`

b) `int arr[];` ✗

c) `int arr[] = new int[3];` ✓

d) `int arr() = new int(3);`

4. Which of the following is the correct way to declare a multidimensional array in Java?

- a) int[] arr; ✗
- b) int arr[[]]; ✗
- c) int[][]arr; ✓
- d) int[[]] arr; ✗

```
public class array
{
    public static void main(String args[])
    {
        int []arr = {1,2,3,4,5};
        System.out.println(arr[2]);
        System.out.println(arr[4]);
    }
}
```

a) 3 and 5

b) 5 and 3

c) 2 and 4

d) 4 and 2

```
public class array
{
    public static void main(String args[])
    {
        int []arr = {1,2,3,4,5};
        System.out.println(arr[5]);
    }
}
```

- a) 4
- b) 5
- c) ArrayIndexOutOfBoundsException
- d) InavlidInputException

7. When does the `ArrayIndexOutOfBoundsException` occur?

- a) Compile-time ✓
- b) Run-time ✓
- c) Not an error ✗
- d) Not an exception at all ✗

8. Which of the following concepts make extensive use of arrays?

- a) Binary trees ✗
- b) Scheduling of processes ✗
- c) Caching ✗
- d) Spatial locality ✓

9. What are the advantages of arrays?

- a) Objects of mixed data types can be stored ~~x~~
- b) Elements in an array cannot be sorted ~~x~~
- c) Index of first element of an array is 1
- d) Easier to store elements of same data type

10. What are the disadvantages of arrays?

- a) Data structure like queue or stack cannot be implemented
- b) There are chances of wastage of memory space if elements inserted in an array are lesser than the allocated size
- c) Index value of an array can be negative
- d) Elements are sequentially accessed

11. Assuming int is of 4bytes, what is the size of int arr[15];

- a) 15
- b) 19
- c) 11
- d) 60

12. In general, the index of the first element in an array is _____

a) 0 ✓

b) -1

c) 2

d) 1

13. Elements in an array are accessed _____

- a) randomly ✓✓
- b) sequentially
- c) exponentially
- d) logarithmically

✓ 1. Process of inserting an element in stack is called _____

- a) Create
- b) Push
- c) Evaluation
- d) Pop

2. Process of removing an element from stack is called _____

- a) Create
- b) Push
- c) Evaluation
- d) Pop

3. In a stack, if a user tries to remove an element from an empty stack it is called _____

- a) Underflow
- b) Empty collection
- c) Overflow
- d) Garbage Collection

4. Pushing an element into stack already having five elements and stack size of 5, then stack becomes _____

- a) Overflow
- b) Crash
- c) Underflow
- d) User flow

5. Entries in a stack are "ordered". What is the meaning of this statement?

- a) A collection of stacks is sortable
- b) Stack entries may be compared with the '<' operation
- c) The entries are stored in a linked list
- d) There is a Sequential entry that is one by one

6. Which of the following is not the application of stack?
- a) A parentheses balancing program
 - b) Tracking of local variables at run time
 - c) Compiler Syntax Analyzer
 - d) Data Transfer between two asynchronous process

7. Consider the usual algorithm for determining whether a sequence of parentheses is balanced. The maximum number of parentheses that appear on the stack AT ANY ONE TIME when the algorithm analyzes: $((()((()((()))))?)$

- a) 1
- b) 2
- c) 3
- d) 4 or more

8. Consider the usual algorithm for determining whether a sequence of parentheses is balanced. Suppose that you run the algorithm on a sequence that contains 2 left parentheses and 3 right parentheses (in some order). The maximum number of parentheses that appear on the stack AT ANY ONE TIME during the computation?

- a) 1
- b) 2
- c) 3
- d) 4 or more

9. What is the value of the postfix expression $6\ 3\ 2\ 4\ +\ -\ *$?

- a) 1
- b) 40
- c) 74
- d) -18

10. Here is an infix expression: $4 + 3 * (6 * 3 - 12)$. Suppose that we are using the usual stack algorithm to convert the expression from infix to postfix notation. The maximum number of symbols that will appear on the stack AT ONE TIME during the conversion of this expression?

- a) 1
- b) 2
- c) 3
- d) 4



2 mins Summary



**TEACHING
WALLAH**

Topic

Data Structure and Algorithm

THANK YOU